Sequential Agentic Workflows using LangGraph

By: Athreya Kidambi

1. Objective:

The objective is to demonstrate how sequential agentic workflows are designed and implemented in real-world enterprise AI systems using LangGraph, Gemini 2.5 Flash, tool integration, and state management.

Two end-to-end workflows were implemented:

i) Telecom SIM Activation & Fraud Verification Workflow

ii) Healthcare Appointment Prioritization Workflow

In both workflows:

- Each node performs a dedicated task
- Tools are used for deterministic evaluations
- Gemini 2.5 Flash is used for reasoning and classification
- Outputs from one step are passed sequentially via shared state to the next step

2. Environment and Setup:

The workflows were implemented and executed in Google Colab using Python.

Key Technologies Used:

- LangGraph for workflow orchestration
- LangChain for LLM and tool abstractions
- Google Gemini 2.5 Flash for reasoning and decision intelligence
- Pydantic for tool input validation
- Mermaid graph rendering for workflow visualization

Model Used:

- gemini-2.5-flash

3. Use Case 1: Telecom SIM Activation & Fraud Verification Workflow

3.1 Business Context:

A telecom provider wants to automate the SIM activation process by:

- Verifying customer identity (KYC)
- Assessing fraud risk
- Making a final activation decision

The workflow must be sequential, where each stage depends on the output of the previous stage.

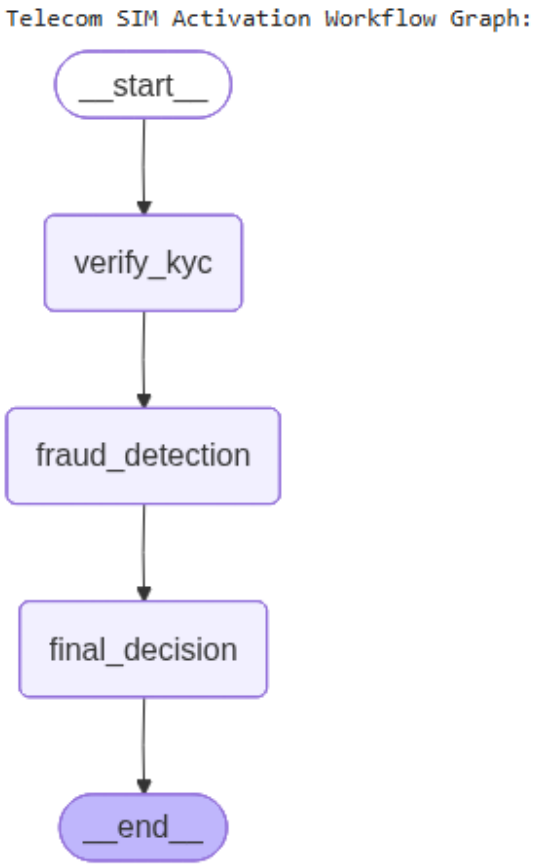
3.2 Workflow Design:

The LangGraph workflow is composed of three sequential nodes, connected linearly using edges.

Sequential Flow

Start → KYC Verification → Fraud Detection → Final Activation Decision → End

A Mermaid visualization of this workflow was generated and captured as evidence.



3.3 Stepwise Implementation:

Step 1: Customer KYC Verification Tool

- Implemented as a LangChain tool using the @tool decorator
- Validates Aadhaar/PAN and document status
- Returns one of:
    - VERIFIED
    - INVALID_DOCUMENT
    - PENDING_VERIFICATION

This tool is invoked inside the first LangGraph node (verify_kyc_node) and stores its output in the shared graph state.

Step 2: Fraud Detection Analysis using Gemini

- Implemented as a separate LangGraph node (fraud_detection_node)
- Uses Gemini 2.5 Flash to analyse:
    - Customer location
    - Number of previous SIM requests
    - KYC status (output from Step 1)

Gemini classifies fraud risk into:

- LOW_RISK
- MEDIUM_RISK
- HIGH_RISK

The classification output is validated and stored in the state for downstream use.

Step 3: Final Activation Decision

- Implemented as the final LangGraph node (final_activation_decision_node)
- Uses:
    - Fraud risk level from Step 2
    - KYC status from Step 1

Decision logic:

- LOW_RISK → SIM Activated
- MEDIUM_RISK → Manual Review
- HIGH_RISK → Reject Application
- Any non-verified KYC automatically results in rejection

The final decision is written to the state and returned as output.

3.4 Execution and Observations:

Multiple test cases were executed to validate different paths through the workflow:

- Low-risk customer with verified documents
- Medium-risk customer with multiple SIM requests
- High-risk customer with excessive SIM attempts
- Customer with invalid documents

Observed Behaviour:

- The workflow executed sequentially with clear step transitions
- KYC output directly influenced fraud analysis
- Fraud risk classification directly influenced final activation decisions
- State propagation was visible through streaming and final outputs

4. Use Case 2: Healthcare Appointment Prioritization Workflow

4.1 Business Context:

A hospital aims to automate patient appointment prioritization by:

- Analysing patient symptoms
- Determining urgency
- Assigning appropriate consultation types

The workflow follows a strict sequential decision-making process.

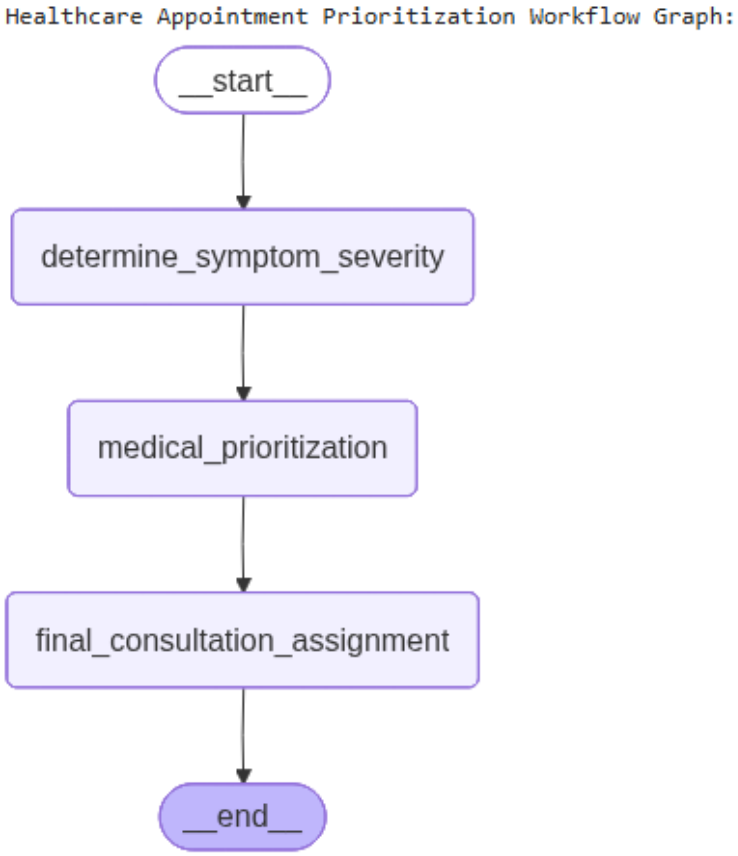
4.2 Workflow Design:

The healthcare workflow is implemented as a three-node sequential LangGraph.

Sequential Flow

Start → Symptom Severity → Medical Prioritization → Consultation Assignment → End

A Mermaid graph visualization was generated and captured.



4.3 Stepwise Implementation:

Step 1: Symptom Severity Tool

- Implemented as a LangChain tool using @tool
- Analyses:
    - Fever
    - Oxygen saturation
    - Heart rate
    - Symptom duration

- Classifies severity into:
    - STABLE
    - MODERATE
    - CRITICAL

The tool output is stored in the workflow state.

Step 2: Gemini Medical Prioritization

- Implemented using Gemini 2.5 Flash
- Evaluates:
    - Symptom severity (from Step 1)
    - Patient age
    - Existing medical conditions

Gemini classifies appointment priority as:

- EMERGENCY
- PRIORITY_CONSULTATION
- REGULAR_CONSULTATION

The output is validated and passed forward via state.

Step 3: Final Consultation Assignment

Based on appointment priority:

- EMERGENCY → ICU/ER
- PRIORITY_CONSULTATION → Specialist Doctor
- REGULAR_CONSULTATION → General Physician

The final assignment is returned as the workflow output.

4.4 Execution and Observations:

Test cases included:

- Critical emergency cases

- Moderate symptoms with comorbidities
- Stable cases
- Edge cases such as young children with moderate symptoms

Observed Behaviour:

- Symptom severity strongly influenced Gemini's prioritization
- Age and comorbidities appropriately escalated urgency
- Final consultation assignment matched clinical expectations
- Sequential dependency between nodes was consistently maintained

5. Conclusion:

This assignment successfully demonstrates the design and implementation of sequential agentic workflows using LangGraph.

Across both use cases:

- Workflows were strictly sequential
- Each node had a clear, well-defined responsibility
- Tools handled deterministic evaluations
- Gemini 2.5 Flash provided reasoning-based classification
- Shared state enabled clean data propagation between steps

These implementations reflect realistic enterprise AI patterns used in domains such as telecom fraud detection and healthcare triage systems.